

The master prompt for the website

STUDYSPARK — COMPLETE PRODUCTION-READY PROMPTS

Copy and use these prompts sequentially to build a full production SaaS product.

TABLE OF CONTENTS

1. [Frontend Landing Page Prompt](#)
 2. [Firebase Setup & Configuration Prompt](#)
 3. [User Authentication System Prompt](#)
 4. [Dashboard & Core Features Prompt](#)
 5. [AI Integration Backend Prompt](#)
 6. [Database Schema & Storage Prompt](#)
 7. [Payment Integration Prompt](#)
 8. [Testing & Quality Assurance Prompt](#)
 9. [Deployment & Production Prompt](#)
 10. [Post-Launch & Monitoring Prompt](#)
-

1. FRONTEND LANDING PAGE

ULTIMATE NO-LIMITS WEBSITE CREATION PROMPT

Create a world-class, cutting-edge landing page for StudySpark — an AI-powered learning platform that acts as every student's personal mentor.

ABSOLUTE FREEDOM — NO BOUNDARIES

You have ZERO RESTRICTIONS. Use ANY technology, framework, library, or approach that will create the most stunning, interactive, and professional result.

Available Technologies — USE ANYTHING:

****Frameworks & Libraries:****

-  **React** — Build with components, hooks, state management
-  **Three.js** — 3D graphics, immersive backgrounds, WebGL
-  **GSAP** — Professional-grade animations

-  Framer Motion — React animation library
-  Particles.js / tsParticles — Particle effects
-  AOS — Animate on scroll
-  Typed.js — Typing animations

****Styling:****

-  Tailwind CSS — Utility-first, rapid development
-  Glassmorphism, Neumorphism — Trendy UI styles
-  CSS Grid & Flexbox — Modern layouts

****Icons & Graphics:****

-  Lucide React — Modern icon library
-  Custom SVGs — Unique graphics

YOUR MISSION

Build something that makes people say: "Wait... this is INSANE. How did they build this?"

Think Like: Apple product launches, Stripe's website, Linear's landing page, Vercel's homepage

Key Principles:

1. 🎨 Visual Impact — Stunning first impression
2. ⚡ Smooth Performance — Buttery 60fps animations
3. 🖱️ Interaction Design — Rewarding micro-interactions
4. 🌊 Flow — Natural, intuitive navigation
5. 💎 Premium Feel — \$100K design quality
6. 🚀 Innovation — Push boundaries, surprise users

COMPLETE WEBSITE STRUCTURE

1. HERO SECTION 🌟

****Goal:**** Instant "WOW" moment

****Elements:****

- Animated 3D background (Three.js floating shapes, particles, or gradient mesh)
- Gradient animated headline with text reveal animation: "Your AI Study Partner. Master Any Subject."
- Compelling sub-headline with typing effect: "Transform hours of reading into minutes of understanding. AI-powered summaries, flashcards, doubt-solving, and personalized study plans."
- Interactive CTA buttons with hover effects:
 - Primary: "Start Learning Free" (glowing, ripple effect)
 - Secondary: "Watch Demo" (ghost button, smooth transition)
- Floating dashboard mockup with 3D tilt on mouse movement

- Scroll indicator — Animated arrow or text

****Advanced Effects:****

- Mouse-follow gradient spotlight
- Parallax layers
- Particle system reacting to cursor
- Smooth scroll snap behavior
- Custom cursor with trail

2. TRUST BAR

- "Trusted by 10,000+ Students"
- "500,000+ Notes Summarized"
- "1M+ Flashcards Created"
- Animated counter that counts up on view
- Smooth horizontal auto-scroll or marquee effect

3. FEATURES SHOWCASE ✨

****6 Feature Cards:****

1.  Smart Summaries — AI condenses notes instantly
2.  AI Doubt Solver — Chat-based learning assistant
3.  Auto Flashcards — Generate from any content
4.  Question Generator — Practice with AI questions
5.  Study Planner — Personalized schedules
6.  Multi-Format — PDFs, images, handwriting

****Design:****

- Bento grid layout (asymmetric, modern)
- Glassmorphism cards with backdrop blur
- Hover effects: Cards lift, glow, tilt slightly
- Icons animate on hover (rotate, scale, morph)
- Stagger animations as cards enter viewport
- Interactive previews

4. HOW IT WORKS

****3-step animated journey****

Steps:

1.  Upload — Drop your study materials
2.  AI Magic — Instant processing & generation
3.  Master — Study with smart tools

****Visualization:****

- Animated SVG path connecting steps
- Progress dots that glow as you scroll

- Morphing illustrations
- Lottie animations for each step icon
- Parallax movement

5. INTERACTIVE PRODUCT DEMO

****Tabbed Interface:****

- Tabs: Summaries | Flashcards | AI Chat | Planner
- Clicking tabs shows animated preview
- Smooth transitions between views
- Realistic UI mockups with hover states

6. BENEFITS SECTION

****Transformation showcase****

Split screen or grid:

Left: Before StudySpark (struggling, overwhelmed)

Right: After StudySpark (confident, organized)

Benefits:

- ⚡ Save 10+ hours weekly
- 🧠 Understand complex topics 3x faster
- ~~100~~ Ace exams with AI practice
- 📚 Never forget what you studied
- 🎯 Stay organized effortlessly
- 🌍 Study anywhere, anytime

7. SOCIAL PROOF

Student testimonials carousel:

- Auto-rotating testimonial cards
- 5-star ratings animated on scroll
- Student avatars
- Quote marks with gradient styling
- Smooth carousel with drag support

8. PRICING / CTA SECTION

"Ready to Transform Your Learning?"

- Free Forever Plan — Highlighted, glowing
- Shows what's included
- Big CTA button with animation
- Trust signals: "No credit card required"

9. FAQ SECTION

Smooth accordion with animations

Questions:

- How does the AI work?
- What formats are supported?
- Is my data private?
- Can I use it offline?
- Is it really free?

10. FOOTER

Sections:

- Product: Features, Pricing, Demo
- Company: About, Blog, Careers
- Resources: Help Center, API Docs, Community
- Legal: Privacy, Terms, Security
- Social: Twitter, LinkedIn, GitHub, Instagram

DESIGN SYSTEM

****Color Palette:****

- Primary: Electric Blue (#0EA5E9, #3B82F6)
- Accent: Cyber Purple (#8B5CF6, #A855F7)
- Success: Neon Green (#10B981, #34D399)
- Background Dark: (#0A0A0F, #1A1A2E)
- Text: (#F1F5F9 on dark, #1E293B on light)

****Typography:****

- Headlines: Inter, Poppins, Space Grotesk (Bold 700-900)
- Body: Inter or System UI (Regular 400-500)

****Animation Timing:****

- Fast: 150-200ms
- Medium: 300-400ms
- Slow: 500-800ms
- Easing: cubic-bezier(0.4, 0, 0.2, 1)

MUST-HAVE EFFECTS

****Scroll-Based:****

- Parallax backgrounds
- Fade-in animations (staggered)
- Scale and rotate on scroll
- Progress indicator
- Snap scrolling between sections

****Hover Effects:****

- 3D card tilts (mouse position)
- Glowing borders
- Scale and lift
- Color shifts
- Ripple effects on buttons

****Background:****

- Animated gradient mesh
- Particle system
- 3D shapes floating
- Spotlight following cursor

DELIVERABLE

Build as a React component artifact with:

- ✓ Fully functional and interactive
- ✓ Smooth 60fps animations
- ✓ Responsive across all devices
- ✓ Clean, commented code
- ✓ Optimized performance
- ✓ Professional, pixel-perfect design

Make this production-ready with proper routing structure for:

- / (landing page)
- /login
- /signup
- /dashboard

Include navigation that will work with React Router.

NOW CREATE SOMETHING LEGENDARY 🚀✨🔥

2. FIREBASE SETUP

🔥 FIREBASE PROJECT SETUP & CONFIGURATION

Create a complete Firebase configuration and setup for StudySpark.

OBJECTIVE

Set up Firebase project with all necessary services for a production SaaS application.

REQUIREMENTS

Use Firebase MCP to:

1. **Create Firebase Project**
 - Project name: studyspark-production
 - Enable Google Analytics
 - Set up billing (Blaze plan for production)
2. **Enable Required Firebase Services:**
 - ☒ Authentication (Email/Password, Google OAuth)
 - ☒ Firestore Database (production mode)
 - ☒ Cloud Storage (for file uploads)
 - ☒ Cloud Functions (for backend logic)
 - ☒ Firebase Hosting (for deployment)
 - ☒ Firebase Analytics
 - ☒ Performance Monitoring
3. **Create Firebase Config File**

Generate a complete `firebase.config.js` with:

```
``javascript
// Firebase configuration
export const firebaseConfig = {
  apiKey: "...",
  authDomain: "...",
  projectId: "...",
  storageBucket: "...",
  messagingSenderId: "...",
  appId: "...",
  measurementId: "..."
};
```

4. Environment Setup

Create `.env.local` file structure:

```
VITE_FIREBASE_API_KEY=
VITE_FIREBASE_AUTH_DOMAIN=
VITE_FIREBASE_PROJECT_ID=
VITE_FIREBASE_STORAGE_BUCKET=
VITE_FIREBASE_MESSAGING_SENDER_ID=
VITE_FIREBASE_APP_ID=
```

VITE_FIREBASE_MEASUREMENT_ID=
VITE_ANTHROPIC_API_KEY=

5. Security Rules

Create Firestore security rules:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    // User data - only owner can read/write
    match /users/{userId} {
      allow read, write: if request.auth != null && request.auth.uid == userId;
    }

    // Notes - only owner can access
    match /notes/{noteId} {
      allow read, write: if request.auth != null &&
        resource.data.userId == request.auth.uid;
    }

    // Flashcards - only owner can access
    match /flashcards/{cardId} {
      allow read, write: if request.auth != null &&
        resource.data.userId == request.auth.uid;
    }

    // Study plans - only owner can access
    match /studyPlans/{planId} {
      allow read, write: if request.auth != null &&
        resource.data.userId == request.auth.uid;
    }
  }
}
```

Create Storage security rules:

```
rules_version = '2';
service firebase.storage {
  match /b/{bucket}/o {
    match /users/{userId}/{allPaths=**} {
      allow read, write: if request.auth != null && request.auth.uid == userId;
    }
  }
}
```



```
}  
}
```

6. Initialize Firebase in Project

Create `src/lib/firebase.js`:

```
import { initializeApp } from 'firebase/app';  
import { getAuth } from 'firebase/auth';  
import { getFirestore } from 'firebase/firestore';  
import { getStorage } from 'firebase/storage';  
import { getAnalytics } from 'firebase/analytics';  
  
// Import config from environment  
const firebaseConfig = {  
  apiKey: import.meta.env.VITE_FIREBASE_API_KEY,  
  authDomain: import.meta.env.VITE_FIREBASE_AUTH_DOMAIN,  
  projectId: import.meta.env.VITE_FIREBASE_PROJECT_ID,  
  storageBucket: import.meta.env.VITE_FIREBASE_STORAGE_BUCKET,  
  messagingSenderId: import.meta.env.VITE_FIREBASE_MESSAGING_SENDER_ID,  
  appId: import.meta.env.VITE_FIREBASE_APP_ID,  
  measurementId: import.meta.env.VITE_FIREBASE_MEASUREMENT_ID  
};  
  
// Initialize Firebase  
const app = initializeApp(firebaseConfig);  
  
// Initialize services  
export const auth = getAuth(app);  
export const db = getFirestore(app);  
export const storage = getStorage(app);  
export const analytics = getAnalytics(app);  
  
export default app;
```

DELIVERABLE

Provide:

1. Complete Firebase project setup instructions
2. All configuration files
3. Security rules for Firestore and Storage

4. Environment variable template
5. Firebase initialization code
6. Setup verification checklist

Use Firebase MCP to create and configure everything.

3. AUTHENTICATION SYSTEM

```markdown

### # 🔒 AUTHENTICATION SYSTEM

Build a complete, production-ready authentication system for StudySpark.

#### ## OBJECTIVE

Create secure user authentication with email/password and Google OAuth.

#### ## FEATURES TO BUILD

##### ### 1. Authentication Context & State Management

Create `src/contexts/AuthContext.jsx`:

- User state management
- Login/logout functionality
- Session persistence
- Protected route logic
- Loading states
- Error handling

##### ### 2. Sign Up Page (`/signup`)

**\*\*Design Requirements:\*\***

- Clean, modern form design
- Email and password inputs
- Password strength indicator
- Terms of service checkbox
- "Sign up with Google" button
- Link to login page
- Real-time validation
- Error messages
- Loading states

**\*\*Functionality:\*\***

- Email/password signup
- Google OAuth signup
- Form validation (email format, password strength)
- Duplicate account detection
- Automatic profile creation in Firestore
- Redirect to onboarding after signup

### ### 3. Login Page (`/login`)

#### **\*\*Design Requirements:\*\***

- Clean, modern form design
- Email and password inputs
- "Remember me" checkbox
- "Forgot password" link
- "Sign in with Google" button
- Link to signup page
- Error messages
- Loading states

#### **\*\*Functionality:\*\***

- Email/password login
- Google OAuth login
- Form validation
- Error handling (wrong credentials, account not found)
- Session management
- Redirect to dashboard after login

### ### 4. Password Reset Flow

#### **\*\*Forgot Password Page:\*\***

- Email input
- Send reset link button
- Success/error messages
- Back to login link

#### **\*\*Reset Password Page:\*\***

- New password input
- Confirm password input
- Password strength indicator
- Submit button
- Success redirect to login

### ### 5. Protected Routes Component

Create `src/components/ProtectedRoute.jsx`:

- Check authentication status
- Redirect to login if not authenticated
- Show loading spinner while checking
- Handle edge cases

### ### 6. User Profile Management

After authentication, create user document in Firestore:

```
```\javascript
{
  uid: "user-id",
  email: "user@example.com",
  displayName: "User Name",
  photoURL: "url",
  createdAt: timestamp,
  lastLogin: timestamp,
  plan: "free",
  settings: {
    theme: "dark",
    notifications: true
  },
  stats: {
    notesSummarized: 0,
    flashcardsCreated: 0,
    questionsGenerated: 0
  }
}
```

AUTHENTICATION HOOKS

Create custom hooks:

src/hooks/useAuth.js

```
// Custom hook for authentication
export const useAuth = () => {
  // Return user, login, logout, signup methods
}
```

src/hooks/useRequireAuth.js

```
// Hook that redirects if not authenticated
export const useRequireAuth = () => {
  // Check auth and redirect
}
```

SECURITY REQUIREMENTS

-  Password minimum 8 characters
-  Email verification (optional but recommended)
-  Secure password hashing (Firebase handles this)
-  HTTPS only in production
-  CSRF protection
-  Rate limiting on login attempts
-  Session timeout after inactivity

ERROR HANDLING

Handle all Firebase auth errors:

- auth/email-already-in-use
- auth/invalid-email
- auth/user-not-found
- auth/wrong-password
- auth/weak-password
- auth/network-request-failed
- auth/too-many-requests

UI/UX REQUIREMENTS

- Smooth page transitions
- Loading states for all async operations
- Clear error messages (user-friendly, not technical)
- Success messages
- Form validation feedback
- Responsive design (mobile-first)
- Accessible (keyboard navigation, screen readers)

DELIVERABLE

Build as React components with:

1. AuthContext with all authentication logic
2. Signup page component
3. Login page component
4. Password reset components
5. Protected route wrapper
6. Custom authentication hooks
7. Proper error handling
8. Loading states
9. Responsive design
10. Integration with Firebase Auth

Make it production-ready, secure, and beautiful.

4. DASHBOARD & CORE FEATURES

```markdown

### # 📊 DASHBOARD & CORE FEATURES

Build the main StudySpark application dashboard and all core features.

#### ## OBJECTIVE

Create a fully functional, beautiful dashboard where students can access all StudySpark features.

#### ## DASHBOARD STRUCTURE

##### ### Main Layout Components:

##### 1. \*\*Sidebar Navigation\*\*

- Logo and branding
- Navigation items:
  - 🏠 Home / Dashboard
  - 📝 My Notes
  - 📇 Flashcards
  - ❓ Practice Questions
  - 📅 Study Planner
  - 💬 AI Chat
  - ⚙️ Settings
- User profile section at bottom
- Collapse/expand functionality
- Active state indicators
- Smooth transitions

## 2. **\*\*Top Navigation Bar\*\***

- Search functionality
- Notifications icon
- User avatar with dropdown menu:
  - Profile
  - Settings
  - Billing
  - Help
  - Logout
- Theme toggle (dark/light)

## 3. **\*\*Main Content Area\*\***

- Dynamic content based on selected section
- Smooth page transitions
- Loading states
- Empty states with helpful CTAs

# ## CORE FEATURES TO BUILD

## ### 1. DASHBOARD HOME (`/dashboard`)

### **\*\*Overview Section:\*\***

- Welcome message with user's name
- Current streak counter
- Quick stats cards:
  - Notes summarized this week
  - Flashcards reviewed today
  - Study hours this month
  - Upcoming study sessions

### **\*\*Quick Actions:\*\***

- Upload new notes button (prominent)
- Start AI chat button
- Create flashcards button
- Generate questions button

### **\*\*Recent Activity:\*\***

- Last 5 notes summarized
- Recently created flashcards
- Recent AI chat sessions
- Click to open/continue

### **\*\*Upcoming Tasks:\*\***

- Study plan items due today
- Scheduled review sessions
- Recommended topics to review

## #### 2. NOTES MANAGER (`/dashboard/notes`)

### **\*\*Features:\*\***

- Upload interface (drag & drop zone)
- Support formats: PDF, images (JPG, PNG), text files
- File size limit indicator
- Upload progress bar
- Processing status

### **\*\*Notes Library:\*\***

- Grid or list view toggle
- Search and filter functionality
- Sort by: date, subject, name
- Each note card shows:
  - Thumbnail/icon
  - Title (editable)
  - Subject/category
  - Upload date
  - File size
- Actions: View, Download, Delete, Generate Flashcards, Ask AI

### **\*\*Note Detail View:\*\***

- Original file preview
- AI-generated summary
- Key concepts highlighted
- Action buttons:
  - Create flashcards from this
  - Generate practice questions
  - Ask AI about this
  - Export as PDF
  - Share

### **\*\*AI Summary Generation:\*\***

- Click "Summarize" button
- Loading animation
- Display structured summary:
  - Main topics
  - Key points
  - Important definitions
  - Formulas



- Diagrams explained
- Edit and save summary

### ### 3. FLASHCARDS SYSTEM (`/dashboard/flashcards`)

#### **\*\*Flashcard Decks:\*\***

- Grid view of all decks
- Create new deck button
- Each deck shows:
  - Title
  - Card count
  - Last studied date
  - Progress percentage
  - Subject/category

#### **\*\*Deck View:\*\***

- List of all cards in deck
- Add new card manually
- Generate cards from notes (AI)
- Bulk actions (delete, move)
- Edit cards inline

#### **\*\*Study Mode:\*\***

- Full-screen flashcard interface
- Flip animation (click to reveal answer)
- Navigation: Next, Previous
- Mark as: Easy, Medium, Hard
- Keyboard shortcuts
- Progress indicator
- Spaced repetition algorithm
- Session summary at end

#### **\*\*Card Creation:\*\***

- Manual creation form
- AI generation from notes:
  - Select source note
  - Choose topic/section
  - Set number of cards
  - Generate button
- Review and edit generated cards

### ### 4. QUESTION GENERATOR (`/dashboard/questions`)

#### **\*\*Question Bank:\*\***

- Browse all generated questions
- Filter by:
  - Subject
  - Difficulty
  - Question type (MCQ, short answer, long answer)
  - Topic
- Search functionality

#### **\*\*Generate Questions:\*\***

- Select source (note or topic)
- Choose question type:
  - Multiple choice
  - Short answer
  - Long answer
  - Numerical problems
- Set difficulty level
- Set quantity (1-50)
- Generate button
- AI processing indicator

#### **\*\*Practice Mode:\*\***

- Quiz interface
- Timer (optional)
- Question navigation
- Answer submission
- Instant feedback (if MCQ)
- Show correct answers
- Explanation for each answer
- Score tracking
- Performance analytics

#### **\*\*Results & Analytics:\*\***

- Score percentage
- Time taken
- Correct/incorrect breakdown
- Topic-wise performance
- Recommendations for improvement
- Save results to history

### **### 5. AI CHAT ASSISTANT (`/dashboard/chat`)**

#### **\*\*Chat Interface:\*\***

- Clean, modern chat UI (like ChatGPT)
- Message history

- Typing indicators
- Code syntax highlighting
- Mathematical formula rendering
- Image support in messages
- Export chat as PDF

**\*\*Features:\*\***

- Ask any study-related question
- Upload image for explanation
- Context awareness (knows user's notes)
- Multi-turn conversations
- Suggested follow-up questions
- Clear conversation button
- New chat button

**\*\*Chat History:\*\***

- Sidebar with previous conversations
- Search through chats
- Star important chats
- Delete chats
- Resume any conversation

**### 6. STUDY PLANNER (`/dashboard/planner`)**

**\*\*Calendar View:\*\***

- Month/week/day views
- Color-coded by subject
- Drag and drop events
- Click to add new event
- Visual progress indicators

**\*\*Study Plan Generator (AI):\*\***

- Input form:
  - Exam date
  - Subjects to cover
  - Available hours per day
  - Difficulty level of topics
  - Current knowledge level
- Generate plan button
- AI creates detailed schedule:
  - Daily breakdown
  - Topic allocation
  - Time estimates
  - Priority marking

- Review sessions scheduled

#### **\*\*Task Management:\*\***

- To-do list view
- Mark as complete
- Reschedule tasks
- Add notes to tasks
- Set reminders

#### **\*\*Analytics:\*\***

- Study time tracking
- Completion rates
- Subject-wise distribution
- Productivity insights
- Streak tracking

### **#### 7. SETTINGS PAGE (`/dashboard/settings`)**

#### **\*\*Profile Settings:\*\***

- Edit display name
- Change profile picture
- Update email
- Change password

#### **\*\*Preferences:\*\***

- Theme (dark/light/auto)
- Default subject categories
- Notification preferences
- Language settings
- Time zone

#### **\*\*Account:\*\***

- Plan details (Free/Pro)
- Usage statistics
- Billing information (if paid)
- Export all data
- Delete account

### **## TECHNICAL REQUIREMENTS**

#### **#### State Management:**

- Use React Context or Redux for global state
- Local state for component-specific data
- Persist user preferences to Firestore

### ### Data Storage (Firestore Collections):

```
```javascript
// users/{userId}
{
  email, displayName, photoURL, plan, settings, stats, createdAt
}

// notes/{noteId}
{
  userId, title, subject, uploadedAt, fileUrl, fileType, fileSize,
  summary, status, thumbnailUrl
}

// flashcards/{deckId}
{
  userId, title, subject, cardCount, lastStudied, createdAt,
  cards: [
    { question, answer, difficulty, timesReviewed, lastReviewed }
  ]
}

// questions/{questionSetId}
{
  userId, sourceNoteId, subject, generatedAt, questionType,
  questions: [
    { question, options, correctAnswer, explanation, difficulty }
  ]
}

// studyPlans/{planId}
{
  userId, title, startDate, endDate, subjects, generatedAt,
  tasks: [
    { title, date, duration, subject, completed, priority }
  ]
}

// chats/{chatId}
{
  userId, title, createdAt, lastMessage,
  messages: [
    { role, content, timestamp }
  ]
}
```

```
]
}
```

File Upload to Firebase Storage:

- Path structure: `users/{userId}/notes/{noteId}/{filename}`
- Generate download URLs
- Implement upload progress
- Handle errors

Performance Optimization:

- Lazy load components
- Implement pagination for large lists
- Use React.memo for expensive components
- Debounce search inputs
- Optimize images
- Code splitting by route

UI/UX REQUIREMENTS

- Responsive design (mobile, tablet, desktop)
- Smooth animations and transitions
- Loading skeletons for async content
- Empty states with helpful CTAs
- Error boundaries for graceful failures
- Toast notifications for actions
- Confirmation dialogs for destructive actions
- Keyboard shortcuts for power users
- Accessibility (ARIA labels, focus management)

DELIVERABLE

Build complete dashboard with:

1. All layout components (sidebar, topbar)
2. All feature pages (notes, flashcards, questions, chat, planner)
3. Full CRUD operations for all data types
4. AI integration placeholder functions
5. File upload and management
6. Responsive design
7. Error handling

8. Loading states
9. Empty states
10. Beautiful UI with animations

Create this as a multi-file React application structure with proper routing.

5. AI INTEGRATION

```markdown

### # AI INTEGRATION — ANTHROPIC CLAUDE API

Integrate Claude AI to power all StudySpark's intelligent features.

#### ## OBJECTIVE

Connect all AI features to Anthropic's Claude API for production-ready functionality.

#### ## FEATURES TO IMPLEMENT

##### ### 1. NOTE SUMMARIZATION

**Endpoint:** `/api/summarize-note``

**Input:**

- Note content (text extracted from PDF/image)
- Subject (optional)
- Desired summary length (short/medium/detailed)

**Claude Prompt:**

You are an expert study assistant. Summarize the following study material for a student.

Material: {noteContent}

Subject: {subject}

Please provide a structured summary with:

1. Main Topics (bullet points)
2. Key Concepts (definitions and explanations)
3. Important Formulas (if any)
4. Critical Points to Remember

## 5. Diagrams/Visuals Explained (if mentioned)

Make it clear, concise, and easy to understand for exam preparation.

**\*\*Output:\*\***

- Structured markdown summary
- Estimated reading time
- Key topics extracted

### ### 2. FLASHCARD GENERATION

**\*\*Endpoint:\*\*** `/api/generate-flashcards`

**\*\*Input:\*\***

- Source content (note summary or topic)
- Number of cards requested
- Difficulty level

**\*\*Claude Prompt:\*\***

Create {numberOfCards} flashcards from the following study material.

Content: {content}

Format each flashcard as JSON: { "question": "...", "answer": "...", "difficulty": "easy|medium|hard", "category": "..." }

Make questions clear and answers comprehensive. Include:

- Definitions
- Concepts
- Formulas
- Important facts
- Applications

Return ONLY valid JSON array.

**\*\*Output:\*\***

- Array of flashcard objects
- Ready to save to Firestore

### ### 3. QUESTION GENERATION



**\*\*Endpoint:\*\*** `/api/generate-questions`

**\*\*Input:\*\***

- Source content
- Question type (MCQ, short answer, long answer, numerical)
- Difficulty level
- Number of questions

**\*\*Claude Prompt for MCQs:\*\***

Generate {numberOfQuestions} multiple-choice questions from this material.

Content: {content}

Difficulty: {difficulty}

Format as JSON: { "question": "...", "options": ["A) ...", "B) ...", "C) ...", "D) ..."], "correctAnswer": "A", "explanation": "...", "topic": "..."} }

Make distractors plausible. Include explanation for correct answer. Return ONLY valid JSON array.

**\*\*Output:\*\***

- Array of question objects with answers
- Explanations included

#### ### 4. AI CHAT ASSISTANT

**\*\*Endpoint:\*\*** `/api/chat`

**\*\*Input:\*\***

- User message
- Conversation history
- User's context (uploaded notes, subjects)

**\*\*Claude System Prompt:\*\***

You are StudySpark AI, an expert study assistant helping students learn better.

The student has access to these notes: {userNotes}

Subjects: {userSubjects}

Your role:

- Explain concepts clearly and simply
- Provide step-by-step solutions
- Give examples and analogies
- Encourage and motivate
- Ask clarifying questions
- Suggest study strategies

Be friendly, patient, and educational. Use markdown for formatting.

**\*\*Features:\*\***

- Streaming responses for real-time feel
- Context-aware (knows user's notes)
- Can reference uploaded materials
- Multi-turn conversations
- Code/math rendering support

#### #### 5. STUDY PLAN GENERATION

**\*\*Endpoint:\*\*** `/api/generate-study-plan`

**\*\*Input:\*\***

- Exam date
- Subjects and topics
- Available study hours per day
- Current knowledge level
- Learning pace preference

**\*\*Claude Prompt:\*\***

Create a personalized study plan for a student.

Details:

- Exam Date: {examDate}
- Days Available: {daysUntilExam}
- Subjects: {subjects}
- Topics per subject: {topics}
- Available hours/day: {hoursPerDay}
- Current level: {knowledgeLevel}

Generate a day-by-day study schedule as JSON: { "totalDays": number, "subjects": [...], "schedule": [ { "day": 1, "date": "YYYY-MM-DD", "tasks": [ { "time": "9:00 AM - 11:00 AM", "subject": "...", "topic": "...", "type": "learn|practice|review", "priority": "high|medium|low", "duration": "2 hours" } ], "totalHours": number } ], "recommendations": [...] }

Balance subjects, include review sessions, build progressively. Return ONLY valid JSON.

**\*\*Output:\*\***

- Complete structured study plan
- Daily breakdowns
- Time allocations
- Priority markings

## ## IMPLEMENTATION STRUCTURE

### ### Create API Service Layer

**\*\*File:\*\*** `src/services/ai.service.js`

```javascript

import Anthropic from '@anthropic-ai/sdk';

```
const anthropic = new Anthropic({
  apiKey: import.meta.env.VITE_ANTHROPIC_API_KEY,
  dangerouslyAllowBrowser: true // For client-side (or use Cloud Functions)
});
```

```
export const aiService = {
  async summarizeNote(noteContent, subject) {
    // Call Claude API
    // Return structured summary
  },
```

```
  async generateFlashcards(content, count, difficulty) {
    // Call Claude API
    // Parse JSON response
    // Return flashcards array
  },
```

```
  async generateQuestions(content, type, difficulty, count) {
    // Call Claude API
    // Parse JSON response
    // Return questions array
  },
```

```
async chat(message, history, userContext) {  
  // Call Claude API with streaming  
  // Return response stream  
},
```

```
async generate
```